

# Solver Ideal

Melhorias ao Sistema · 08/08/2026

---

**Tenho material excelente e diretamente aplicável — incluindo projetos que correm exatamente no cenário do Morgan (7-8 agentes Python num Mac Mini). Tenho tudo o que preciso para o relatório.**

---

---

## Relatório de Investigação — Detecção Proactiva de Erros Silenciosos

---

**Missão:** tornar o Solver do Morgan proactivo (detectar antes de reportado)

**Contexto-alvo:** 8 agentes Python 3.12 num Mac Mini · €0–20/mês · foco em erros silenciosos

---

## Respostas às perguntas de investigação

---

### 1. Como os melhores SRE/AIOps detectam erros ANTES de reportados?

Não esperam por exceções. Modelam o **comportamento normal** (baseline) e disparam quando há **desvio persistente**. O padrão dominante em produção é *anomaly detection sobre séries temporais + validação de persistência* (distinguir spike transitório de degradação sustentada). O Datadog Watchdog e o FBDetect da Meta (7 anos em produção, 800k séries temporais, apanha regressões de 0.005%) são a prova cabal disto.

### 2. Que padrões de "proactive scanning" existem hoje?

Três famílias reais e documentadas: **(a) anomaly detection** (z-score/EWMA/MAD sobre métricas), **(b) canary analysis** (Google/Waze/iFood — comparar versão nova vs. baseline antes de promover, auto-abort), **(c) chaos engineering** (Flipkart/Intertech com LitmusChaos — injetar falhas para validar deteção). Para o Morgan, (a) é o mais relevante; (b) e (c) são pesados demais (exigem Kubernetes).

### 3. Que bibliotecas Python fazem scan proactivo?

Encontrei várias reais e *zero-infra*: `trace-sentinel`, `blackice`, `drift-sre`, `pipeline-sentinel`, `si-fleet-health-monitor`, e — crucialmente — `observeco`, `clevagent` e `axme` que são feitas **especificamente para frotas de agentes LLM**.

### 4. O que Datadog/Sentry/Honeycomb/OTel fazem de diferente dos reactivos?

O reactivo (Solver atual do Morgan) age **quando alguém reporta**. Estes: Datadog/Watchdog fazem **baseline ML contínuo + forecasting**; Sentry faz **fingerprinting/grouping + regressão de issues** (deteta issue *novo* ou *que reapareceu*); OpenTelemetry modela exceções como `span events (exception.type/message/stacktrace)` permitindo capturar erros **em qualquer span** sem `try/except` explícito. A diferença essencial: **eles observam o fluxo todo continuamente, não só o ponto onde alguém gritou "erro"**.

### 5. Padrão mais leve e eficaz para 8 agentes Python num Mac Mini?

**Heartbeat/watchdog com verificação de *work-progress* + z-score/EWMA sobre métricas, tudo em SQLite local.** É o que a `observeco` faz num M4 Mac Mini com 7 agentes — o cenário quase idêntico ao Morgan.

---

## Padrões encontrados (por ordem de relevância)

**1. Heartbeat + Work-Progress Watchdog** | Cada agente emite pulso periódico (30–60s). Distingue-se *liveness* (processo vivo) de *work-progress* (agente realmente a produzir output). Estados: HEALTHY → DEGRADED → UNHEALTHY/UNKNOWN. Apanha o pior erro silencioso: agente "verde" mas parado (deadlock, lock, retry loop, exceção engolida). | `pip install observeco / axme / si-fleet-health-monitor` (zero-dependency, SQLite local, dashboard ASCII/htmx) | €0 | [observeco](https://github.com/observeco/observeco) (7 agentes M4 Mac Mini), [AXME](https://github.com/AxmeAI/ai-agent-heartbeat-monitoring), [fleet-health-monitor](https://github.com/SuperInstance/fleet-health-monitor)

**2. Anomaly Detection com Persistência (z-score/EWMA/MAD + filtro de ruído)** | Aprende baseline online por métrica (Welford/EWMA), calcula desvio em  $\sigma$ , mas só reporta se o desvio **persistir** N pontos. Filtra 80–90% do ruído transitório → evita alert fatigue. O mesmo princípio do FBDetect da Meta. | `pip install blackice` (RegimeDetector, O(1) memória, determinístico) ou `trace-sentinel` (statistical + semantic, stdlib puro) ou `drift-sre` (CUMSUM/EWMA + webhook) | €0 | [blackice](https://pypi.org/project/blackice/), [trace-sentinel](https://pypi.org/project/trace-sentinel/), [FBDetect/Meta SOSP'24](https://dl.acm.org/doi/10.1145/3785504)

**3. Detecção Semântica de Novidade em Logs** | Além de números anômalos, deteta **linhas de log cujo conteúdo é novo** (nunca visto no baseline). Ideal para erros silenciosos que não crasham mas produzem mensagens estranhas. Usa centróide decaído do "normal", pontua novidade em  $\sigma$ . | `trace-sentinel` com `enable_semantic=True` (opcional numpy) | €0 | [trace-sentinel](https://pypi.org/project/trace-sentinel/) |

**4. Circuit Breaker + Auto-Heal com cooldown** | N falhas → *trip* → bloqueia agente → cooldown → auto-restart (máx 3 tentativas / 5 min). Impede cascata de falhas silenciosas entre os 8 agentes. Regra de ouro: **"verify before claiming, fail closed"** — nunca auto-remediar o que pode mandar mensagem a cliente. | `observeco` (circuit breaker + heal button embutidos) ou `custom-agent-harness` (self-heal por camadas) | €0 | [observeco](https://github.com/observeco/observeco), [custom-agent-harness](https://github.com/yash-gadodia/custom-agent-harness) |

**5. OpenTelemetry span events → captura passiva de exceções** | Instrumenta `span.record_exception()`; exceções viajam como `span events` com `exception.type/message/stacktrace`. Captura erros em **qualquer** span sem `try/except` no sítio certo. Desde Dez/2025 o Sentry Python tem `OTLPIntegration` que converte estes eventos em issues automaticamente. | `opentelemetry-sdk` + OTLP local (porta 4318) → SQLite ou Sentry free tier | €0 (self-host) a ~€20/mês (Sentry team) | [Sentry OTLP](https://docs.sentry.io/concepts/otlp/), [OTLP error tracking](https://urgentry.com/guides/observability/ot